Universidad de Las Palmas de Gran Canaria

Degree in Data Science and Engineering

Big Data

Assignment 1

Basic Matrix Multiplication in Python, Java and C

Student: María Báez Marrero

Academic Year 2026–2027

Abstract

This assignment compares basic dense matrix multiplication implementations in Python, Java and C using the same standard triple-loop algorithm and identical input matrices.

Execution time is measured for matrix sizes from 50 to 250 using five repetitions per configuration, summarized with the median and interquartile range. Peak Resident Set Size (Peak RSS) is used as the memory metric.

Correctness is verified using known test cases, identity and zero matrices, cross-language checksums, and NumPy as a trusted reference implementation.

At $n = 250$, the median execution time was approximately 1.06 s for Python, 12.7 ms for Java and 19.5 ms for C. Peak RSS at the same size was approximately 21.4 MB for Python, 55.9 MB for Java and 2.1 MB for C.

The results show that the three implementations have the same theoretical $O(n^3)$ algorithmic structure but substantially different observed runtime and memory behaviour under the tested execution environment.

The estimated log-log slopes were approximately 3.02 for Python, 3.15 for C and 2.10 for Java, showing behaviour close to the expected cubic model for Python and C over the tested range.

1 Introduction

Matrix multiplication is a fundamental operation in scientific computing, data processing and machine learning. Although the underlying algorithm may be identical, its practical performance can differ considerably depending on the programming language, runtime environment, memory representation and compiler or interpreter.

The objective of this assignment is to compare straightforward dense matrix multiplication implementations written in Python, Java and C under controlled experimental conditions.

The research question is:

> How do execution time and memory usage differ when the same basic dense matrix multiplication algorithm is implemented in Python, Java and C on the same hardware and using the same input matrices?

The hypothesis is that execution time will increase approximately according to the cubic growth of the standard matrix multiplication algorithm as matrix size increases. Differences are also expected between the three implementations because Python, Java and C use different execution and memory models.

2 Background

The standard dense matrix multiplication algorithm computes each element of the result matrix as

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

For two square matrices of size $n \times n$, the straightforward triple-loop algorithm performs arithmetic work proportional to n^3, giving a theoretical time complexity of $O(n^3)$ [1].

Practical performance, however, depends not only on asymptotic complexity but also on the execution environment, memory representation, compiler optimizations and runtime behaviour.

Java benchmarking requires particular care because Just-in-Time compilation, garbage collection and other JVM effects can influence measurements [2]. More generally, seemingly minor experimental choices can introduce performance-measurement bias [3].

For Python measurements, `time.perf_counter()` is used because it provides a high-resolution performance counter suitable for measuring short durations [4].

3 Methodology

3.1 Implementations

The matrix multiplication algorithm is implemented in Python, Java and C using the same loop order:

$$i \rightarrow j \rightarrow k.$$

No optimized matrix multiplication libraries such as NumPy or BLAS are used in the benchmarked implementations.

Python stores matrices as lists of lists of Python integers. Java uses two-dimensional `int[][]` arrays. C uses dynamically allocated matrices with integer pointers.

The implementations are sequential and do not explicitly use multithreading.

3.2 Input Data

The tested matrix sizes are:

$$50,\ 100,\ 150,\ 200,\ 250.$$

All matrix values are integers between 0 and 9.

The matrices are generated once using Python with random seed 42 and stored as CSV files. The same input files are read by Python, Java and C so that all implementations process identical data.

With the selected value range and maximum size, the maximum possible value of one output element is

$$250 \times 9 \times 9 = 20250,$$

which remains well within the integer range used by Java and C.

3.3 Correctness Validation

Correctness is checked using three types of tests:

- matrices with a known multiplication result,
- multiplication by the identity matrix,
- multiplication by the zero matrix.

The implementations also calculate a checksum of the resulting matrix after each timed execution. For every matrix size, the checksum is identical in Python, Java and C.

In addition, the Python implementation is compared against NumPy's matrix multiplication as a trusted reference implementation. All tested matrix sizes produced matching results.

3.4 Runtime Measurement

Each matrix size is executed five times for each language.

Input loading and allocation of the output matrix are excluded from the timed region.

Inside the timed region, each output element is initialized and the standard multiplication kernel is executed.

The checksum is calculated after the timer stops, so checksum computation does not affect the measured runtime.

The timers used are:

- Python: `time.perf_counter()`,
- Java: `System.nanoTime()`,
- C: `CLOCK_MONOTONIC`.

Java performs three untimed warm-up executions for each matrix size before the measured repetitions.

The C implementation is compiled using Apple Clang with optimization level `-O2`.

For every language and matrix size, the median execution time and interquartile range (IQR) are reported.

3.5 Memory Measurement

Memory usage is measured using Peak Resident Set Size (Peak RSS).

On macOS, Peak RSS is collected using:

```
/usr/bin/time -l
```

The metric represents the peak memory usage of the complete process. Therefore, it includes runtime overhead such as the Python interpreter and the Java Virtual Machine.

One Peak RSS measurement is recorded for every language and matrix size.

3.6 Experimental Environment

The experiments were executed on a MacBook Pro with:

- Apple M4 processor,
- 10 CPU cores,
- 16 GB RAM,
- macOS 26.3.1,
- ARM64 architecture.

Software versions include:

- Python 3.13.0,
- OpenJDK 25.0.2,
- Apple Clang 21.0.0,
- pandas 3.0.0,
- matplotlib 3.10.8.

The final experimental scope is limited to matrix sizes up to $n = 250$. A practical runtime budget of 5 seconds per individual multiplication and a Peak RSS budget of 1 GB are used as feasibility limits.

Although the predefined runtime budget was 5 seconds per multiplication, the experiment was intentionally limited to $n = 250$ in order to keep the comparison common across all three languages and to maintain a compact and inspectable dataset. Therefore, $n = 250$ is the largest tested size in this study, not the computational limit of the machine.

4 Results

4.1 Correctness Results

All three implementations produced identical checksums for every tested matrix size. In addition, the complete result matrices produced by the Python implementation matched NumPy exactly. Java and C were validated through the same input matrices, correctness tests and matching cross-language checksums.

Table 1: Cross-language checksum validation and NumPy reference comparison.

n	Checksum	NumPy match
50	2580329	Yes
100	20043070	Yes
150	68563585	Yes
200	161354779	Yes
250	315567802	Yes

Because the experiment uses integer arithmetic and all values remain within the representable range, correctness comparison is exact and no numerical tolerance is required.

4.2 Execution Time

Table 2 summarizes the median execution time and IQR for every language and matrix size.

Table 2: Median execution time and interquartile range in seconds.

n	Py Med.	Py IQR	Java Med.	Java IQR	C Med.	C IQR
50	0.008169	0.000141	0.000432	0.000009	0.000069	2.0×10^{-9}
100	0.067900	0.015727	0.001090	0.000194	0.000691	0.000077
150	0.231970	0.004558	0.003211	0.000288	0.002257	0.000227
200	0.543361	0.008861	0.006495	0.000168	0.009325	0.000052
250	1.055285	0.028924	0.012666	0.000114	0.019843	0.000311

4.3 Comparison with the Cubic Model

To compare the measurements with the expected $O(n^3)$ model, the median execution time was normalized by n^3.

Table 3: Median execution time normalized by n^3.

n	Python	Java	C
50	6.53×10^{-8}	3.46×10^{-9}	5.52×10^{-10}
100	6.79×10^{-8}	1.09×10^{-9}	6.91×10^{-10}
150	6.87×10^{-8}	9.51×10^{-10}	6.69×10^{-10}
200	6.79×10^{-8}	8.12×10^{-10}	1.17×10^{-9}
250	6.75×10^{-8}	8.11×10^{-10}	1.27×10^{-9}

For Python, the normalized execution time remains almost constant across the tested sizes. This is consistent with the expected cubic growth of the standard triple-loop algorithm.

C also remains within approximately the same order of magnitude, although the normalized cost increases for the largest tested sizes.

Java shows a different pattern. Its normalized execution time decreases substantially between the smallest and largest tested matrices, indicating that runtime effects influence the measurements for small matrix sizes.

The log-log slope is estimated using

$$p = \frac{\ln(T_{250}/T_{50})}{\ln(250/50)},$$

where T_n is the median execution time for matrix size n.

A simple log-log slope estimate between the smallest and largest tested sizes gives approximately 3.02 for Python, 3.52 for C and 2.10 for Java. Python and C are therefore close to the theoretical cubic exponent of 3 over the tested range. The lower Java slope indicates that the observed Java timings are affected by additional runtime effects, particularly for the smaller matrix sizes.

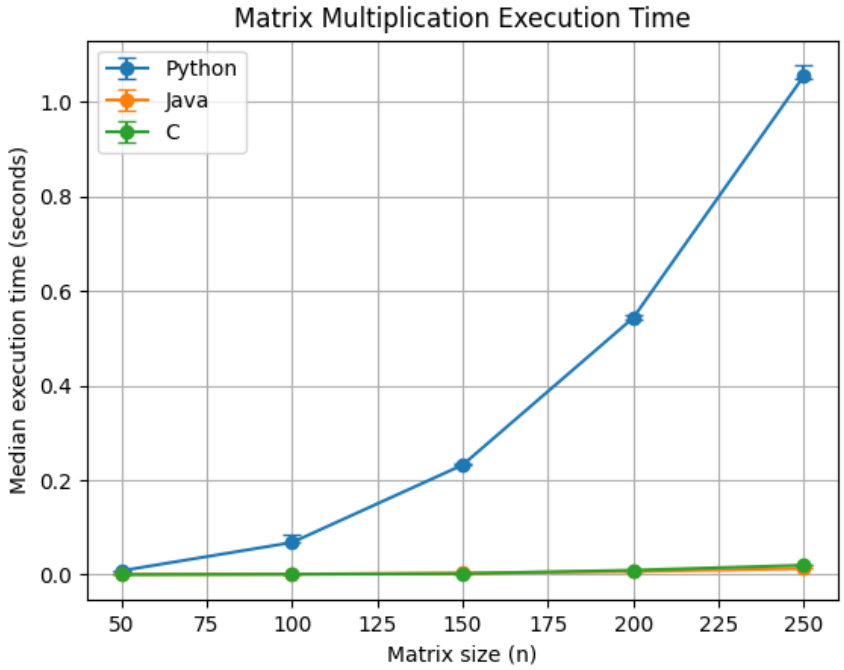


Figure 1: Median execution time over five measured repetitions for Python, Java and C. Error bars represent the interquartile range.

Because Python execution times are much larger than those of Java and C, a logarithmic scale helps distinguish the lower execution times more clearly.

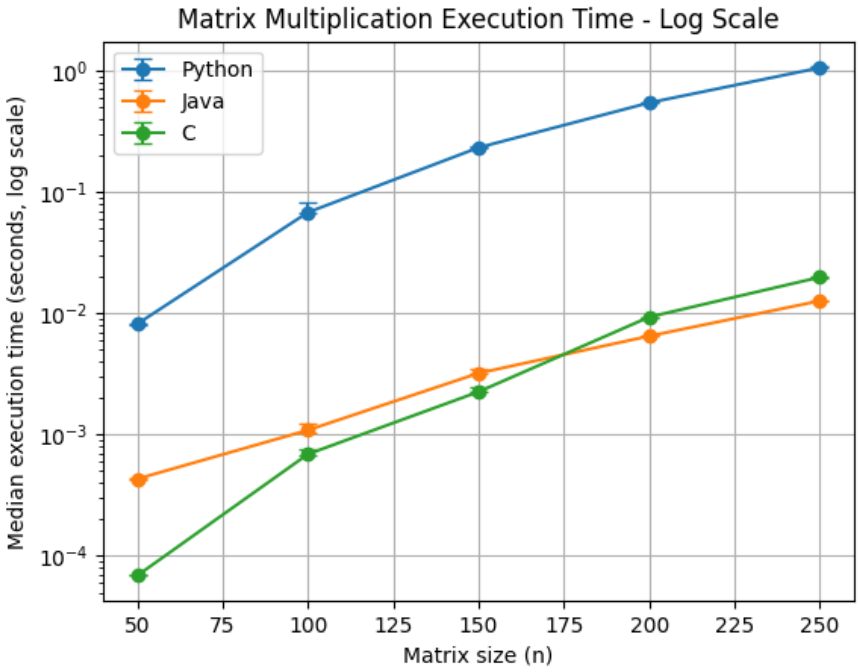


Figure 2: Median execution time over five measured repetitions using a logarithmic vertical scale. Error bars represent the interquartile range.

4.4 Memory Usage

Table 4 summarizes the measured Peak RSS values.

Table 4: Peak RSS memory usage in MB (10^6 bytes).

n	Python	Java	C
50	17.42	43.02	1.33
100	17.99	44.73	1.46
150	18.83	46.84	1.61
200	20.04	51.30	1.85
250	21.41	55.93	2.08

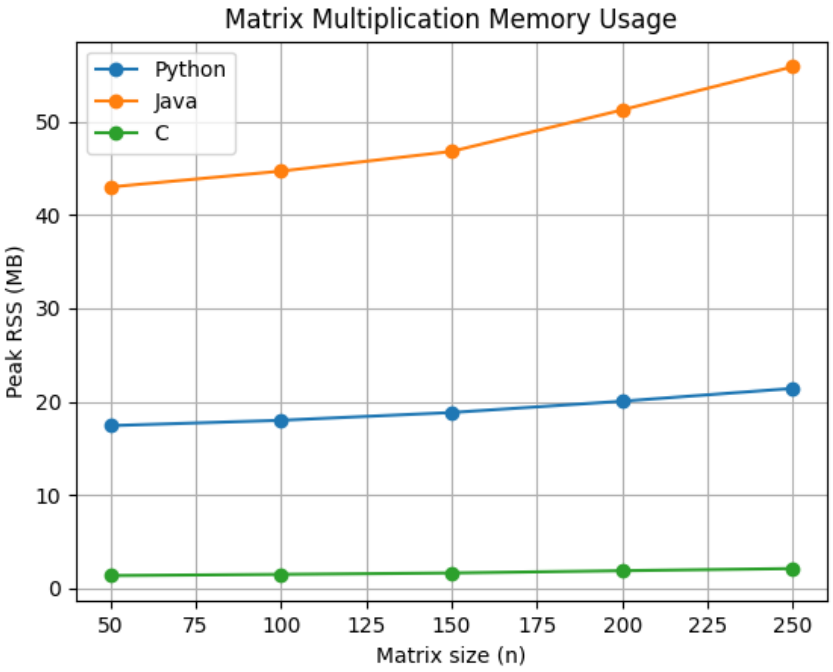


Figure 3: Peak RSS memory usage for the three implementations.

5 Discussion

The results show substantial runtime differences between the three implementations.

Python is consistently slower than Java and C for every tested matrix size. In Python, matrices are stored as lists of references to Python integer objects rather than as contiguous primitive integer arrays. This introduces additional interpreter and object-management overhead and can reduce memory locality.

Java and C show much smaller execution times and are closer to each other. C has the lowest median execution time for the smallest tested sizes, while Java produces lower median times for the larger tested sizes of 200 and 250. These observations are specific to the tested hardware, compiler settings, JVM version and implementation and should not be interpreted as universal language rankings.

The fact that Java becomes faster than C for the largest tested sizes is an observed result, but its exact cause was not isolated in this experiment. Possible explanations include differences in JIT optimization, array handling and memory-access behaviour. Further experiments would be required to determine the cause.

The normalized runtime analysis provides additional evidence about the growth trend. Python shows an almost constant normalized execution time and an estimated log-log slope of approximately 3.02, which is very close to the theoretical cubic exponent. C produces a higher estimated slope of approximately 3.52. This still reflects strong superlinear growth, but the deviation from 3 indicates that constant factors and memory-access effects influence the measurements over the limited tested range.

The increase in normalized cost observed for C at the largest sizes may also be influenced by cache and memory-access effects, particularly because matrix B is accessed by columns in the chosen $i \to j \to k$ loop order. This was not measured directly and is therefore only a possible explanation.

Java behaves differently over the tested range, with an estimated slope of approximately 2.10. The smallest Java measurements have a relatively high normalized cost, while the values stabilize

for larger matrices. This suggests that JVM runtime effects and JIT compilation influence the measurements at small sizes. Therefore, the Java measurements should not be interpreted as a clean empirical estimate of the asymptotic exponent over this limited range.

The Peak RSS measurements show a different pattern from the execution-time results.

Between $n = 50$ and $n = 250$, measured Peak RSS increased from approximately 17.4 MB to 21.4 MB for Python, from 43.0 MB to 55.9 MB for Java, and from 1.33 MB to 2.08 MB for C.

This corresponds to an increase of approximately 4.0 MB for Python, 12.9 MB for Java and 0.75 MB for C.

The C increase is close to the theoretical storage increase expected for three dense matrices containing 32-bit integers. At $n = 250$, three matrices require approximately

$$3 \times 250^2 \times 4 = 750000$$

bytes of element storage.

Python and Java show substantially larger process-level memory usage because Peak RSS measures the complete process rather than only the matrix data. The Python measurement includes interpreter and object-management overhead. The Java result includes JVM, heap and runtime overhead, and may also include temporary objects created while parsing the CSV input files.

Therefore, these Peak RSS values should not be interpreted as direct measurements of matrix storage alone.

The experiment also has several limitations. Only one computer and one software environment are tested. Memory is measured once for each configuration rather than repeatedly. The matrix sizes are limited to 250, and different runtime environments introduce unavoidable asymmetries such as JIT compilation in Java and ahead-of-time optimization in C.

6 Conclusions

This experiment compared basic dense matrix multiplication implementations in Python, Java and C using shared inputs and the same triple-loop algorithm.

All implementations produced correct and equivalent results, confirmed through basic correctness tests, cross-language checksums and comparison with NumPy.

The measured execution times show that Python is substantially slower than Java and C for the tested configurations. Java and C exhibit much lower execution times, with their relative performance varying across matrix sizes.

Peak RSS measurements show that Java uses the largest amount of process-level memory, while C uses the least in the tested environment. These differences reflect not only matrix storage but also runtime and process overhead.

Overall, the results demonstrate that identical algorithmic complexity does not imply identical practical performance. Language runtime, memory representation, compiler optimization and execution environment all influence observed behaviour.

The study does not determine why Java becomes faster than C for the largest tested sizes, nor does it establish behaviour for substantially larger matrices. These questions remain open for future experiments.

7 Reproducibility Materials

The complete project is available at:

`https://github.com/mariabaezz/big-data-assignment1`

The repository contains the source code, shared input matrices, raw CSV measurements, analysis scripts, figures, environment information and README. The README includes the exact commands required to reproduce the experiments and figures.

Use of Generative AI

Claude was used as a support tool for explaining concepts, planning the experimental workflow, debugging, and revising the English wording of the documentation and report.

References

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2022.

[2] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous java performance evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications*, pages 57–76, 2007.

[3] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong. In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XIV)*, pages 265–276, 2009.

[4] Python Software Foundation. time — time access and conversions. `https://docs.python.org/3/library/time.html`. Accessed 13 October 2026.